



Guía para conseguir el 100% en el "Hito 2"

El Hito 2 marca el inicio del desarrollo visual de sus proyectos. El objetivo de esta etapa es abandonar los datos estáticos locales y **demostrar dominio sobre la construcción de interfaces dinámicas, el tipado estricto en TypeScript, la manipulación segura del DOM y la gestión de peticiones asíncronas con feedback continuo.**

Para alcanzar el puntaje máximo (10/10 - Nivel "Excelente"), su repositorio de frontend (Vite + TypeScript) debe cumplir estrictamente con tres pilares. Aquí les explicamos exactamente qué buscará el evaluador y cómo lograrlo.



Pilar 1: Modelado de Datos en TypeScript (3 Puntos)

El objetivo: Definir contratos de datos herméticos que restrinjan los tipos de entrada y los estados de la interfaz de usuario, garantizando previsibilidad en toda la arquitectura.

Checklist para el Nivel "Excelente":

- **Cero Uso de any:** Está estrictamente prohibido recurrir al tipado dinámico (any). Si una variable, respuesta de API o argumento de función usa any, perderán puntos en la compilación estricta.
- **Control de Estados con enum:** Queda prohibido controlar estados críticos (ej. estados de una orden, disponibilidad o roles) con cadenas de texto libres sueltas (string). Se deben definir e importar enumeraciones estrictas (enum).
- **Interfaces Estructuradas:** Los archivos .ts en la carpeta models/ deben definir interfaces (interface) puras exportadas para cada entidad de su negocio, con tipos primitivos claros y sin propiedades desorganizadas.

TypeScript

```
// src/models/booking.ts
```

```
// 1. Enumeración estricta para el control de estados de la UI
```

```
export enum BookingStatus {  
  PENDING = "PENDING",  
  CONFIRMED = "CONFIRMED",  
  CANCELLED = "CANCELLED"  
}
```

```
// 2. Modelo de datos hermético (Cero 'any')
```

```
export interface Booking {  
  id: string;  
  customerName: string;  
  quantity: number;  
  status: BookingStatus; // Uso explícito del enum  
}
```

Pilar 2: Manejo del DOM y Formularios (3 Puntos)

El objetivo: Capturar nodos del árbol visual del navegador sin provocar excepciones nulas en la consola y procesar interacciones mediante componentes modulares y validaciones reactivas.

Checklist para el Nivel "Excelente":

- **Guardias de Nulidad (Type Guards):** No asuman que un elemento siempre existe en el HTML. Al capturar nodos con `document.getElementById`, deben aplicar validaciones contra nulos (`if (elemento !== null)`) antes de mutar su contenido.
- **Neutralización de Formularios:** Todo escuchador de eventos (`addEventListener("submit", ...)`) debe ejecutar `event.preventDefault()` como su primera instrucción para evitar que la página recargue su estado.
- **Aserciones Especializadas:** Extraigan datos de formularios aplicando la palabra clave `as` hacia el componente HTML específico (ej: `as HTMLInputElement`) para leer propiedades como `.value` de forma tipada.

TypeScript

// src/main.ts (Captura de Eventos y Manejo del DOM)

```
const bookingForm = document.getElementById("form-reserva") as HTMLFormElement | null;
```

```
if (bookingForm !== null) {  
  bookingForm.addEventListener("submit", (event: Event) => {  
    // 1. Interceptar el envío nativo del navegador  
    event.preventDefault();
```

```
    // 2. Extraer elementos con aserción de tipo especializada  
    const nameInput = document.getElementById("txt-nombre") as HTMLInputElement;  
    const quantityInput = document.getElementById("txt-cantidad") as HTMLInputElement;  
    const errorBlock = document.getElementById("bloque-error");
```

```
    const nameValue = nameInput.value.trim();  
    const quantityValue = parseInt(quantityInput.value);
```

```
// 3. Validaciones reactivas en el cliente
if (nameValue.length === 0 || isNaN(quantityValue) || quantityValue <= 0) {
  if (errorBlock) errorBlock.textContent = "Error: Ingrese datos válidos.";
  return;
}

if (errorBlock) errorBlock.textContent = "";
console.log("Payload extraído de forma limpia:", { nameValue, quantityValue });
});
}
```

Pilar 3: Arquitectura Asíncrona y Bloques de Control (4 Puntos)

El objetivo: Conectar la interfaz con servicios externos utilizando las sintaxis modernas de asincronía, asegurando inmunidad ante caídas de red y guiar al usuario mediante indicadores visuales.

Checklist para el Nivel "Excelente":

- **Sintaxis Moderna async/await:** Estructuren el consumo de datos remotos mediante funciones async en lugar de cadenas de promesas anidadas (.then()).
- **Control con Bloques try/catch:** Inmunicen las llamadas fetch envolviéndolas en un bloque try/catch. Es obligatorio validar la propiedad response.ok para interceptar respuestas 404 o 500 y forzar la entrada al bloque catch.
- **Feedback Continuo en el DOM:** Inyecten un indicador visual de carga (ej: <p>Cargando datos...</p>) en el contenedor antes de disparar la llamada de red, y muestren un mensaje de error descriptivo en la pantalla si la red o el servidor fallan.

TypeScript

```
// src/main.ts (Consumo Asíncrono)
import { Booking } from "../models/booking";
import { generateBookingCardHtml } from "../components/BookingCard";
```

```
async function loadCatalog(): Promise<void> {
  const container = document.getElementById("contenedor-catalogo");
  if (!container) return; // Guardia de nulidad
```

```
// 1. Feedback visual de carga antes de la petición
container.innerHTML = "<p class='loading'>Cargando eventos desde el servidor...</p>";
```

```

try {
  // 2. Disparar llamada asíncrona Fetch
  const response = await fetch("https://api.ejemplo.com/v1/bookings");

  // 3. Validar el canal HTTP (Response OK)
  if (!response.ok) {
    throw new Error(`Error del Servidor: Código HTTP ${response.status}`);
  }

  // 4. Mapear y tipar el JSON a la interfaz correspondiente
  const data = (await response.json()) as Booking[];

  container.innerHTML = "";
  if (data.length === 0) {
    container.innerHTML = "<p>No hay eventos disponibles en este momento.</p>";
    return;
  }

  // 5. Renderizado seguro
  data.forEach(booking => {
    container.innerHTML += generateBookingCardHtml(booking);
  });

} catch (error: any) {
  // 6. Captura de errores con feedback visual en pantalla
  console.error("Fallo crítico de red:", error);
  container.innerHTML = `
    <div class="alerta-error">
      <p>No fue posible obtener los datos de la cartelera.</p>
      <small>${error.message}</small>
    </div>
  `;
}

document.addEventListener("DOMContentLoaded", loadCatalog);

```



El Entregable: Un README.md Profesional

Su repositorio publicado en GitHub debe incluir un archivo README.md claro y conciso en la raíz con:

1. **Título y Descripción:** Nombre del proyecto integrador autónomo y una breve explicación de la solución interactiva.
2. **Tecnologías:** Especificar el uso de TypeScript Vanilla, Vite y módulos nativos.
3. **Comandos de Instalación y Ejecución:**
 - Para instalar dependencias: `npm install`
 - Para ejecutar el servidor de desarrollo en caliente: `npm run dev`
 - Para ejecutar la verificación de tipos de TypeScript: `npm run build`

¡Mucho éxito con la entrega! Aseguren el tipado hermético, la captura segura del DOM y la buena experiencia de usuario ante llamadas asíncronas antes de enviar su enlace.